

Firebase

Firebase Authentication

Firebase Cloud Firestore

Firebase Cloud Messaging

Firebase

- To incorporate *Firebase* into a project...
 - Access the *Firebase*
 - Authenticate and access the console
 - <https://console.firebase.google.com/>
 - Create project
 - Give a unique name
 - Include *Google Analytics*
 - Associate *Android application*
 - Associate the SHA-1 key (some services may require this information)
 - Download the `google-services.json` file
 - Place the file in the project's `app` folder

Firebase

- Include the configurations indicated in the two `build.gradle` files

- `build.gradle.kts` (project)

```
plugins {  
    ...  
    id("com.google.gms.google-services") version "4.4.4" apply false  
}
```

- `build.gradle.kts` (module)

- *Plugins*

```
plugins {  
    ...  
    id("com.google.gms.google-services")  
}
```

- In the dependencies

```
implementation(platform("com.google.firebase:firebase-bom:34.6.0"))  
implementation("com.google.firebase:firebase-analytics-ktx")  
implementation("com.google.firebase:firebase-auth-ktx")  
implementation("com.google.firebase:firebase-firestore-ktx")  
implementation("com.google.firebase:firebase-storage-ktx")  
implementation("com.google.firebase:firebase-messaging-ktx")  
//implementation("com.google.android.gms:play-services-auth:21.2.0")
```

Firebase Authentication

- In *Firebase*, add the desired authentication methods to use in the application
 - In the context of this lesson, choose:
 - *Email/password*
 - *Google*

Authentication

- Create variable to hold the reference to the service
`private lateinit var auth : FirebaseAuth`
- In onCreate
`auth = FirebaseAuth`
- Get the authenticated user using `auth.currentUser`
 - Example:

```
override fun onStart () {  
    super.onStart ()  
    showUser(auth.currentUser)  
}  
  
fun showUser(user: FirebaseAuthUser?) {  
    val str = when (user) {  
        null -> "Not authenticated user "  
        else -> " User : ${user.email}"  
    }  
  
    binding.tvStatus.text = str  
    Log.i(TAG, str)  
}
```

Authentication with email/password

- Function to register/create user based on *email*

```
fun createUserWithEmail(email: String, password: String) {  
    auth.createUserWithEmailAndPassword(email, password)  
        .addOnSuccessListener(this) { result ->  
            Log.i(TAG, "createUser: success")  
            showUser(auth.currentUser)  
        }  
        .addOnFailureListener(this) { e ->  
            Log.i(TAG, "createUser: failure ${e.message}")  
            showUser(null)  
        }  
}
```

Authentication with email/password

- Function to verify the user based on *email*

```
fun signInWithEmail(email: String, password: String) {  
    auth.signInWithEmailAndPassword(email, password)  
        .addOnSuccessListener(this) { result ->  
            Log.d(TAG, "signInWithEmail: success")  
            showUser(auth.currentUser)  
        }  
        .addOnFailureListener(this) { e->  
            Log.d(TAG, "signInWithEmail: failure ${e.message}")  
            showUser(null)  
        }  
}
```

Authentication with email/password

- Function to *Logout*

```
fun signOut() {  
    if (auth.currentUser != null) {  
        auth.signOut()  
    }  
    showUser(auth.currentUser)  
}
```


Exercise

- Get the `FAuthUtil` class, where the essential code for these operations is already implemented
 - <https://shorturl.at/udn4v>
- Put the class in an appropriate package
 - Example: `<package_base>.utils`
- Create a `ViewModel` class to assist with application data management

Authentication with Google account

- Create a variable for the Google sign in client

```
private lateinit var googleSignInClient: GoogleSignInClient
```

- Create the customer in onCreate

```
val gso = GoogleSignInOptions.Builder(GoogleSignInOptions.DEFAULT_SIGN_IN)
    .requestIdToken(default_web_client_id)
    .requestEmail()
    .build()
googleSignInClient = GoogleSignIn.getClient(this, gso)
```

- Define the *string* `default_web_client_id` copying the *string* directly from the *Firebase console* into the *Google Sign in provider* settings

Authentication with Google account

- Use the following code to verify the success of the Google authentication process and to proceed to the Firebase authentication

```
val signInWithGoogle = registerForActivityResult(  
    ActivityResultContracts.StartActivityForResult()  
) { result ->  
    val task = GoogleSignIn.getSignedInAccountFromIntent(result.data)  
    try {  
        val account = task.getResult(ApiException::class.java)!!  
        Log.d(TAG, "firebaseAuthWithGoogle:" + account.id)  
        firebaseAuthWithGoogle(account.idToken!!)  
    } catch (e: ApiException) {  
        Log.i(TAG, "onActivityResult - Google authentication: failure")  
    }  
}
```

- To initiate Google authentication

```
signInWithGoogle.launch(googleSignInClient.signInIntent)
```

Authentication with Google account

- Get the *Google Sign In* token and start *Firebase* authentication

```
private fun firebaseAuthWithGoogle(idToken: String) {  
    val credential = GoogleAuthProvider.getCredential(idToken, null)  
    auth.signInWithCredential(credential)  
        .addOnSuccessListener(this) { result ->  
            Log.d(TAG, "signInWithCredential:success")  
            showUser(auth.currentUser)  
        }  
        .addOnFailureListener(this) { e ->  
            Log.d(TAG, "signInWithCredential:failure ${e.message}")  
            showUser(auth.currentUser)  
        }  
}
```

Authentication with Google account

- To sign out:

```
fun signOutGoogle() {  
    if (auth.currentUser != null) {  
        googleSignInClient.signOut()  
            .addOnCompleteListener(this) {  
                showUser(null)  
            }  
    }  
}
```

Cloud Firestore

- Store data (NoSQL)
 - Data organization:
 - *Collections*
 - *Documents*
 - Attribute-value pairs
- Start a *Firestore* database in the *Firebase console* in a suitable region (e.g.: *eur3*)
 - In the context of this lesson the test mode should be used
- A data schema can be created in advance in the *Firebase console*

Cloud Firestore

- To add a document

```
fun addDataToFirestore() {  
    val db = Firebase.firestore  
  
    val scores = hashMapOf(  
        "nrgames" to 0,  
        "topscore" to 0  
    )  
    db.collection("Scores").document("Level1").set(scores)  
        .addOnSuccessListener {  
            Log.i(TAG, "addDataToFirestore: Success")  
        }  
        .addOnFailureListener { e->  
            Log.i(TAG, "addDataToFirestore: ${e.message}")  
        }  
}
```

Cloud Firestore

- To change a document

```
fun updateDataInFirestore() {  
    val db = Firebase.firestore  
    val v = db.collection("Scores").document("Level1")  
  
    v.get(Source.SERVER)  
        .addOnSuccessListener {  
            val exists = it.exists()  
            Log.i(TAG, "updateDataInFirestore: Success? $exists")  
            if (!exists)  
                return@addOnSuccessListener  
            val value = it.getLong("nrgames") ?: 0  
            v.update("nrgames", value+1)  
        }  
        .addOnFailureListener { e ->  
            Log.i(TAG, "updateDataInFirestore: ${e.message}")  
        }  
}
```


Cloud Firestore

- To change a document using transactions

```
fun updateDataInFirestoreTrans() {  
    val db = Firebase.firestore  
    val v = db.collection("Scores").document("Level1")  
  
    db.runTransaction { transaction ->  
        val doc = transaction.get(v)  
        if (doc.exists()) {  
            val newnrgames = (doc.getLong("nrgames") ?: 0) + 1  
            val newtopscore = (doc.getLong("topscore") ?: 0) + 100  
            transaction.update(v, "nrgames", newnrgames)  
            transaction.update(v, "topscore", newtopscore)  
            null  
        } else  
            throw FirebaseFirestoreException(  
                "Error",  
                FirebaseFirestoreException.Code.UNAVAILABLE  
            )  
    }.addOnSuccessListener {  
        Log.i(TAG, "updateDataInFirestoreTrans: Success")  
    }.addOnFailureListener { e ->  
        Log.i(TAG, "updateDataInFirestoreTrans: ${e.message}")  
    }  
}
```

Cloud Firestore

- To delete a document

```
fun removeDataFromFirestore() {  
    val db = Firebase.firestore  
    val v = db.collection("Scores").document("Level1")  
  
    v.delete()  
        .addOnSuccessListener {  
            Log.i(TAG, "removeDataFromFirestore: Success")  
        }  
        .addOnFailureListener { e ->  
            Log.i(TAG, "removeDataFromFirestore: ${e.message}")  
        }  
}
```

Cloud Firestore

- To express interest in the changes

```
fun startObserver() {  
    val db = Firebase.firestore  
    db.collection("Scores").document("Level1")  
        .addSnapshotListener { docSS, e ->  
        if (e!=null) {  
            return@addSnapshotListener  
        }  
        if (docSS!=null && docSS.exists()) {  
            val nrgames = docSS.getLong("nrgames")  
            val topscore = docSS.getLong("topscore")  
            Log.i(TAG, "$nrgames : $topscore")  
        }  
    }  
}
```

Cloud Firestore

- To turn off the *listener*, the `ListenerRegistration` object returned by `addSnapshotListener` must be stored, on which the `remove()` operation can be performed

```
var listenerRegistration : ListenerRegistration? = null
```

```
fun startObserver() {  
    stopObserver()  
    val db = Firebase.firestore  
    listenerRegistration = db  
        .collection("Scores")  
        .document("Level1")  
        .addSnapshotListener { docSS, e ->  
            ...  
        }  
}
```

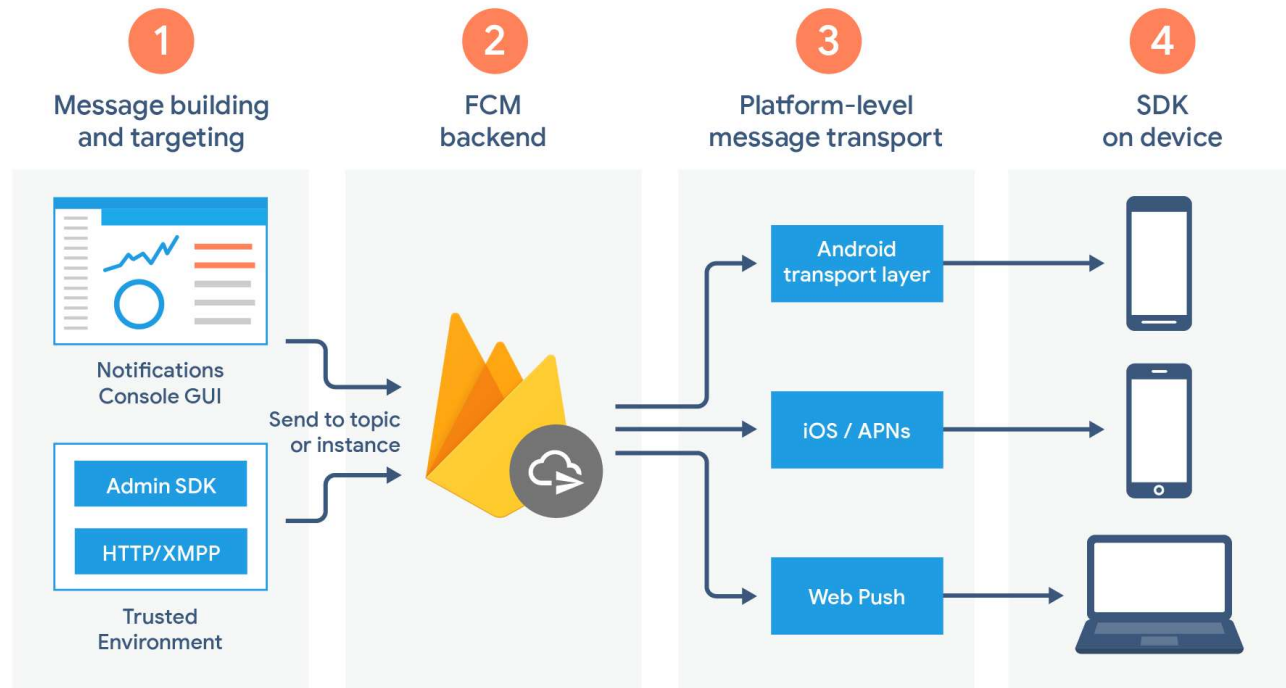
```
fun stopObserver() {  
    listenerRegistration?.remove()  
}
```

Exercise

- Get the `FStorageUtil` class, where the essential code for these operations is already implemented
 - <https://shorturl.at/EfvZA>
- Put the class in an appropriate package
 - Example: `<package_base>.utils`
- Update the `ViewModel` class with the calls to the *Firestore* operations

Firebase Cloud Messaging (FCM)

- The FCM *enables* sending and receiving *Push Notifications*
- Messages are received asynchronously, even if the application is not active
- The application does not need to implement mechanisms that actively check for new messages



Firestore Cloud Messaging

- Received messages can be
 - Notifications
 - Data
 - Both

```
{
  "message": {
    "token": "bk3RNwTe3H0:CI2k_HHwgIpoDKCIZvvDMExUdFQ3P1...",
    "notification": {
      "title": "Portugal vs. Denmark",
      "body": "great match!"
    }
  }
}
```

```
{
  "message": {
    "token": "bk3RNwTe3H0:CI2k_HHwgIpoDKCIZvvDMExUdFQ3P1...",
    "data": {
      "Nick" : "Mario",
      "body" : "great match!",
      "Room" : "PortugalVSDenmark"
    }
  }
}
```

```
{
  "message": {
    "token": "bk3RNwTe3H0:CI2k_HHwgIpoDKCIZvvDMExUdFQ3P1...",
    "notification": {
      "title": "Portugal vs. Denmark",
      "body": "great match!"
    },
    "data" : {
      "Nick" : "Mario",
      "Room" : "PortugalVSDenmark"
    }
  }
}
```

Firebase Cloud Messaging

- By default, message processing is done as follows:

App state	Notification	Data	Both
Foreground	<code>onMessageReceived</code>	<code>onMessageReceived</code>	<code>onMessageReceived</code>
Background	System tray	<code>onMessageReceived</code>	Notification: system tray Data: in extras of the intent.

Firebase Cloud Messaging

- The `onMessageReceived` method is part of the service to be created that derives from the class: `FirebaseMessagingService`
- Register it in the manifest with the `intent-filter`

```
<service android:name=".MyFirebaseMessagingService"
        android:exported="true">
    <intent-filter>
        <action android:name="com.google.firebase.MESSAGING_EVENT" />
    </intent-filter>
</service>
```

Firebase Cloud Messaging

- `FirebaseMessagingService` example:

```
class MyFirebaseMessagingService : FirebaseMessagingService() {
    companion object {
        private const val TAG = "MyFMS"
    }
    override fun onMessageReceived(rMsg: RemoteMessage) {
        super.onMessageReceived(rMsg)
        if (rMsg.data.isNotEmpty()) {
            Log.i(TAG, "onMessageReceived - Data: " +
                rMsg.data.toString())
        }

        rMsg.notification?.run {
            Log.i(TAG, String.format(
                "onMessageReceived - Notification: %s %s %s %s %s",
                title, body, icon, imageUrl, channelId)
            )
        }
    }
}
```

Firebase Cloud Messaging

- If the application is not active, when selecting the notification the application is opened and the application receives the data in the associated `Intent` object
 - Example to list the values stored in the `Intent`

```
intent.extras?.apply {  
    for(k in keySet()) {  
        Log.i(TAG, "Extras: $k -> ${get(k)}")  
    }  
}
```

Topics

- Messages can be sent in the context of certain topics

- To subscribe to a topic:

```
Firestore.messaging.subscribeToTopic("amov")  
    .addOnCompleteListener(this) {  
        if (it.isSuccessful)  
            Log.i(TAG, "AMov topic subscribed")  
    }
```

- To unsubscribe from a topic:

```
Firestore.messaging.unsubscribeFromTopic("amov")
```